INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DISCIPLINA: Padrões de Projeto em Aplicações Web
PROFESSOR: Éder Pereira
ATIVIDADE PRÁTICA: Padrão Estrutural Facade

Contextualização do Problema

Imagine uma pessoa planejando suas férias de verão. Ela pode pesquisar e comprar a passagem aérea, reservar o hotel e agendar passeios individualmente. No entanto, coordenar todos esses sistemas de forma independente é trabalhoso e propenso a erros. As agências de viagens surgem para simplificar essa tarefa, atuando como uma interface única (intermediário) que interage com companhias aéreas, hotéis e empresas de turismo.

Em desenvolvimento de software, o padrão estrutural **Facade** (Fachada) possui a mesma finalidade: prover uma interface simplificada para um conjunto complexo de interfaces ou subsistemas.

No nosso exemplo prático, ao registrar um novo pedido em um e-commerce, o sistema precisa acionar diversos subsistemas: o módulo de **Estoque** (para agendar o envio), o módulo **Financeiro** (para emitir a fatura) e o módulo de **Pós-Venda** (para agendar contatos de satisfação). Em vez do código cliente gerenciar chamadas a todos esses módulos individualmente, criamos uma fachada (`PedidoFacade`) que centraliza esse fluxo e reduz o acoplamento do sistema.

Roteiro de Implementação

1. **Criação do Projeto:**
    - Crie um projeto Java no Eclipse chamado `Facade`.
2. **Modelagem das Classes de Domínio e Subsistemas:**
    - Modele a classe `Pedido.java` com seus atributos encapsulados, construtor e métodos acessores (getters).
    - Modele as classes dos subsistemas (`Estoque.java`, `Financeiro.java` e `PosVenda.java`) definindo os atributos e as assinaturas dos métodos que representam os comportamentos solicitados.

```java
public class Pedido {
    private String produto;
    private String cliente;
    private String enderecoDeEntrega;

    public Pedido(String produto, String cliente, String enderecoDeEntrega) {
        this.produto = produto;
        this.cliente = cliente;
        this.enderecoDeEntrega = enderecoDeEntrega;
    }

    public String getProduto() { return produto; }
    public String getCliente() { return cliente; }
    public String getEnderecoDeEntrega() { return enderecoDeEntrega; }
}
```

```java
public class Estoque {
    // TODO: Definir os atributos necessários para a classe Estoque

```

```
    // Comportamento: Calcular a data de entrega (ex: 2 dias uteis) e exibir a mensagem de confirmacao do envio do produto para o endereco fornecido.
    public void enviaProduto(String produto, String enderecoDeEntrega) {
        // TODO: Implementar o comportamento de envio e estimativa de entrega
    }
}

public class Financeiro {
    // TODO: Definir os atributos necess rios para a classe Financeiro

    // Comportamento: Processar o faturamento exibindo os dados da fatura (cliente e produto).
    public void fatura(String cliente, String produto) {
        // TODO: Implementar o comportamento de faturamento
    }
}

public class PosVenda {
    // TODO: Definir os atributos necess rios para a classe PosVenda

    // Comportamento: Agendar contato futuro com o cliente (ex: 30 dias apos a compra) para verificar a satisfa  o com o produto.
    public void agendaContato(String cliente, String produto) {
        // TODO: Implementar o comportamento de agendamento de pos-venda
    }
}
```

3. **Criação da Classe Fachada (`PedidoFacade`):**

   - Crie a classe `PedidoFacade.java` definindo os atributos para os subsistemas e implementando o método responsável por orquestrar o registro do pedido.

```
public class PedidoFacade {
    // TODO: Definir os atributos para armazenar as referencias dos subsistemas (Estoque, Financeiro, PosVenda)

    public PedidoFacade(Estoque estoque, Financeiro financeiro, PosVenda posVenda) {
        // TODO: Inicializar os atributos da fachada via construtor
    }

    // Comportamento: Orquestrar as chamadas sequenciais para Estoque, Financeiro e PosVenda a partir do Pedido recebido.
    public void registraPedido(Pedido p) {
        // TODO: Invocar o envio do produto no Estoque
        // TODO: Invocar a geracao da fatura no Financeiro
        // TODO: Invocar o agendamento no PosVenda
    }
}
```

4. **Testes do Sistema (`TestePedidoFacade`):**

   - Desenvolva a classe de testes criando instâncias dos subsistemas, da fachada e executando o fluxo de registro de um pedido.

```
public class TestePedidoFacade {
    public static void main(String[] args) {
        // TODO: Instanciar os subsistemas (Estoque, Financeiro, PosVenda)
```

```
4           // TODO: Instanciar a fachada PedidoFacade passando os subsistemas
                criados
5           // TODO: Instanciar um objeto Pedido com dados de teste
6           // TODO: Executar o metodo registraPedido atrav s da fachada
7       }
8   }
```